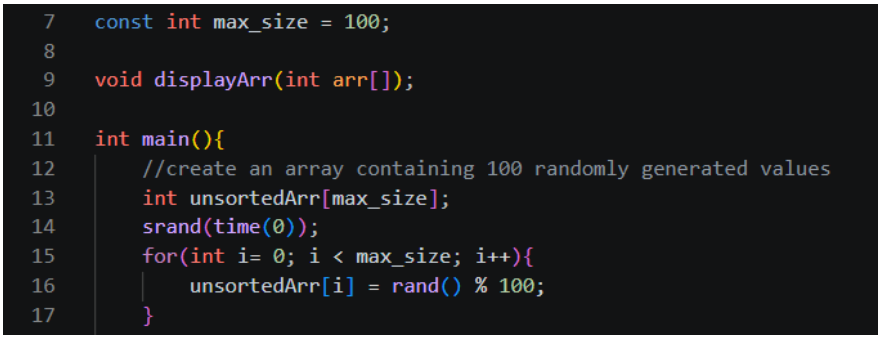
COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	9/8/2026	**DATE SUBMITTED:**	9/10/2026
NAME:	Gabriel Immanuel R. Serrano	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

Code + Console Screenshot	
Observations	The unsortedArray contains 100 randomly generated values that range from 0 to 99 using rand() % 100, which limits the generated values to 99. The srand(time(0)) function makes the generated values different each time the program runs. Then, a for loop is used to fill all 100 indexes of the array with the randomly generated values.

```
7   const int max_size = 100;
8
9   void displayArr(int arr[]);
10
11  int main(){
12      //create an array containing 100 randomly generated values
13      int unsortedArr[max_size];
14      srand(time(0));
15      for(int i= 0; i < max_size; i++){
16          unsortedArr[i] = rand() % 100;
17      }
```

Table 7-1. Array of Values for Sort Algorithm Testing

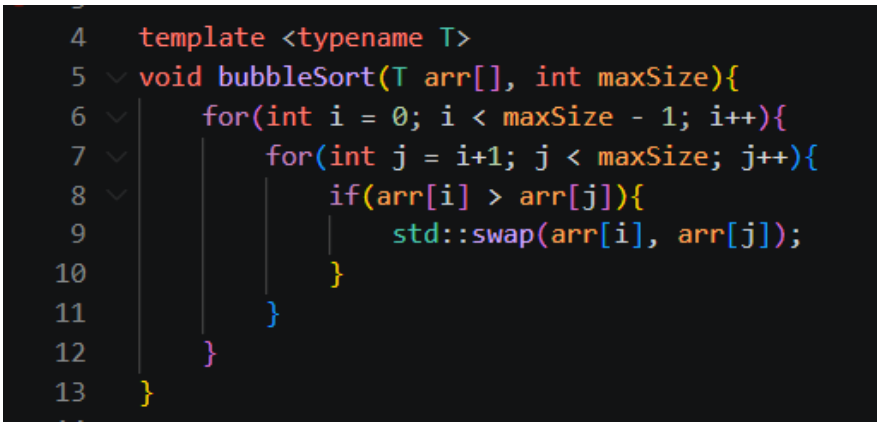
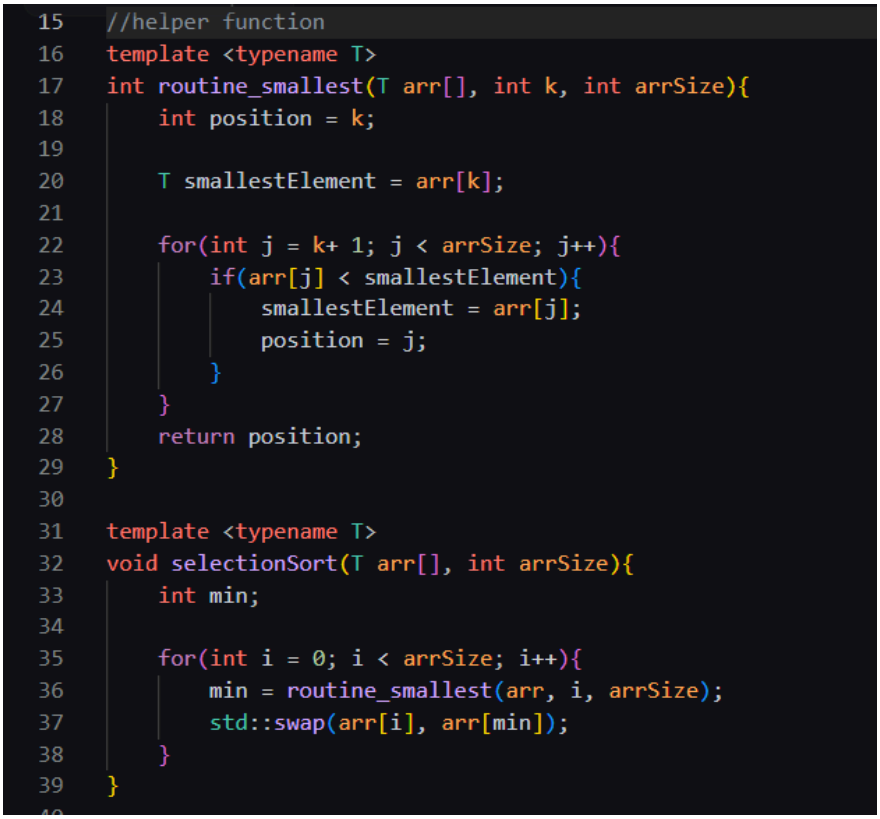
Code + Console Screenshot	
Observations	In the bubbleSort function, we used two for loops to compare the elements of the array and arrange them in ascending order. It works by having the outer loop check an element, while the inner loop compares it with the elements after it. Now if arr[i] is greater than arr[j] we swap their values. The time complexity of this code is O(n^2) because it is a nested loop where both loops run to n times, and there is no exit or return inside the loop so it will keep running until the condition is met.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	
Observations	In the selectionSort function, we created a helper function to find the smallest element's position in the array. It uses a for loop to compare the elements and changes the position if it finds another smaller value. Now the selectionSort function uses a for loop to call the helper function to find the smallest element for each position, then swaps it with the current position and arranges the array. The time complexity of this code is O(n^2) because of the helper function using a for loop to search through the element for each iteration on the selectionSort.

```
15  //helper function
16  template <typename T>
17  int routine_smallest(T arr[], int k, int arrSize){
18      int position = k;
19
20      T smallestElement = arr[k];
21
22      for(int j = k+ 1; j < arrSize; j++){
23          if(arr[j] < smallestElement){
24              smallestElement = arr[j];
25              position = j;
26          }
27      }
28      return position;
29  }
30
31  template <typename T>
32  void selectionSort(T arr[], int arrSize){
33      int min;
34
35      for(int i = 0; i < arrSize; i++){
36          min = routine_smallest(arr, i, arrSize);
37          std::swap(arr[i], arr[min]);
38      }
39  }
```

Table 7-3. Selection Sort Algorithm

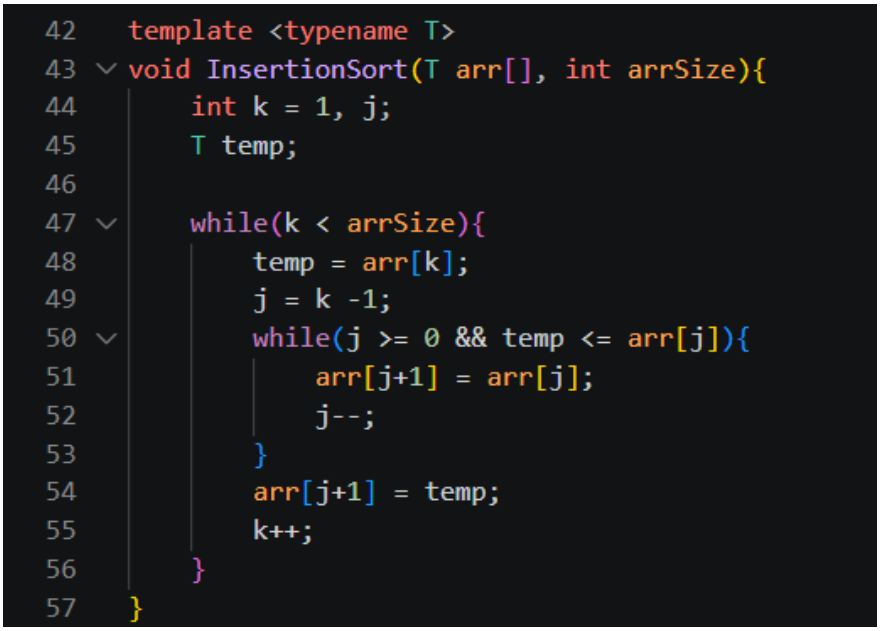
Code + Console Screenshot	
Observations	In the InsertionSort function, the first element of the array is considered sorted already, so the sorting starts from the second element. It uses a while loop instead of a for loop to go through the elements of the array. We declare a temp variable to store the current element, while the inner loop compares it with the element before it. Now, if the temp is smaller than the previous element, the previous element is moved to the right until the temp is placed correctly. The time complexity for this code is O(n^2) because the outer while loop runs through all the elements, while the inner loop compares the elements and moves them to the right.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode

```
START
   DECLARE array size of 100
   DECLARE 5 candidates

   FOR i = 0 to 100
      GENERATE random vote from 1 to 5
      STORE vote in the array
   END FOR

   DISPLAY "Unsorted votes"
   DISPLAY the array

   DISPLAY "Sorted votes"
   SORT the array using the selection sort
   DISPLAY the array

   DECLARE currentVote as the first element of the array
   DECLARE count as 1

   FOR i = 1 to 100
      IF the current element is equal to currentVote
         INCREMENT count
      ELSE
         DISPLAY "Candidate" currentVote "received" count "votes."
         DECLARE currentVote to the current element
         DECLARE count to 1
      END IF
   END FOR

   DISPLAY the total votes for each candidate
END
```

Header File

SortingAlgo > C sortingSupp.h > ...

```h
#ifndef SORTINGSUPP_H
#define SORTINGSUPP_H

//helper function
template <typename T>
int routine_smallest(T arr[], int k, int arrSize){
    int position = k;

    T smallestElement = arr[k];

    for(int j = k+ 1; j < arrSize; j++){
        if(arr[j] < smallestElement){
            smallestElement = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], int arrSize){
    int min;

    for(int i = 0; i < arrSize; i++){
        min = routine_smallest(arr, i, arrSize);
        std::swap(arr[i], arr[min]);
    }
}
```

```h
template <typename T>
void countVotes(T arr[], int arrSize){
    T currentVote = arr[0];
    int count = 1;

    for(int i = 1; i < arrSize; i++){
        if(arr[i] == currentVote){
            count++;
        } else {
            std::cout << "Candidate " << currentVote << " received " << count << " votes." << std::endl;
            currentVote = arr[i];
            count = 1;
        }
    }
    std::cout << "Candidate " << currentVote << " received " << count << " votes." << std::endl;
}

template <typename T>
void displayArray(T arr[], int arrSize){
    for(int i = 0; i < arrSize; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}


#endif
```

Main File

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingSupp.h"

const int maxSize = 100;
const int numCandidates = 5;

int main(){
    int votes[maxSize];
    std::string candidates[numCandidates] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustín",
        "Deja Jayla Bañaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

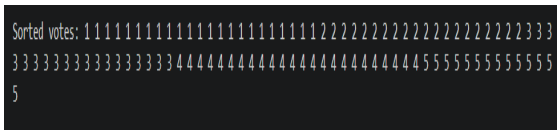
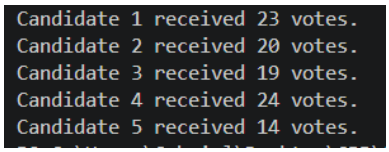
    srand(time(0));
    for(int i = 0; i < maxSize; i++){
        votes[i] = rand() % numCandidates + 1;
    }

    std::cout << "Unsorted votes: ";
    displayArray(votes, maxSize);
    std::cout << std::endl;

    std::cout <<"Sorted votes: ";
    selectionSort(votes, maxSize);
    displayArray(votes, maxSize);
    std::cout << std::endl;

    countVotes(votes, maxSize);

    return 0;
}
```

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
	1 = 23 2 = 20 3 = 19 4 = 24 5 = 14	

Question: Was your developed vote counting algorithm effective? Why or why not?

- Yes, the developed vote counting algorithm was effective because doing a sorting algorithm first made all the votes for the same candidate grouped together, allowing the use of the countVotes() function to count the votes in a linear way instead of checking the array repeatedly. I chose Selection Sort because it uses the routine_smallest() helper function, which finds the smallest value before swapping it into the correct position. I believe this is an optimal way because we are only dealing with values from 1 to 5. The time complexity of Selection Sort is $O(n^2)$ because it uses nested loops to compare the elements, while the voting counting algorithm is just O(n) because all the values are sorted before we count them.

Conclusion :
I learned how to implement a sorting algorithm and how this sorting algorithm can be used to organize a large number of array values. Through this activity, I learned the proper usage of Bubble Sort, Insertion Sort, and Selection Sort. For example, Bubble Sort compares adjacent elements and swaps them, Insertion Sort takes an element and places it in its proper position assuming the first index is already sorted, while Selection Sort finds the smallest element and swaps it into the correct position. Now for the supplementary activity, I found it challenging but not in a hard way since we already have a sorting algorithm and we just need to implement the counting function. After trying and understanding how to do the supplementary I was finally able to do it by creating a for loop that iterates through the sorted array and counts the number of times that number appears for each candidate. Overall, I think I did a great job in this activity because I managed to create and implement the sorting algorithm correctly and not just hard coding it. Also this activity made me realize that understanding the concept is better when it comes to sorting algorithms because it is really easy to get lost when you can't follow how the code iterates through the array.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly & ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction and Code assistances**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **ChatGPT and grammarly was used to correct my grammar and ChatGPT also helped me in debugging and guiding how to create the count votes function**

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.